

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №7.

Тема: «Шаблоны классов»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 10

Выполнила: студентка группы РИС-25-26

Абрамова Валерия Андреевна

Принял: доц. Полякова О.А.

Пермь, 2026

Постановка задачи

Класс-контейнер: МНОЖЕСТВО (SET) с элементами типа int.

Перегружаемые операции:

- [] – доступ к элементу по индексу;
- () – определение размера множества;
- – разность множеств.

Пользовательский класс Money для работы с денежными суммами.

Число должно быть представлено двумя полями:

тип long – рубли;

тип int – копейки.

Анализ задачи

1. Создаём класс Money для хранения денежной суммы в рублях и копейках. Добавляем метод normalize для нормализации (приводим копейки в диапазон 0-99, корректируем рубли). Перегружаем операторы сравнения == и !=, присваивания =, ввода >> и вывода <<.
2. Создаём шаблонный класс Set для хранения множества элементов произвольного типа T. Добавляем приватные поля: динамический массив elements, размер size и вместимость capacity. Добавляем приватный метод contains для проверки наличия элемента.
3. Создаём конструкторы: по умолчанию с capacity=10, с заданной вместимостью, копирования. Создаём деструктор для освобождения памяти.
4. Перегружаем операторы: оператор () возвращает размер множества, оператор [] обеспечивает доступ по индексу с проверкой границ, оператор = для присваивания.
5. Создаём метод add для добавления элемента (только если его ещё нет в множестве). При переполнении массива увеличиваем capacity вдвое.
6. Перегружаем оператор - для вычисления разности множеств (элементы из первого, отсутствующие во втором).
7. Перегружаем операторы ввода >> и вывода << для работы с множеством.
8. В функции main: тестируем Set с типами int, float, double, Money. Для каждого типа создаём два множества, вводим их с клавиатуры, выводим на экран, вычисляем разность. Демонстрируем работу копирования и присваивания.

Код

```

#include <iostream>
#include <stdexcept>
#include <locale>
#include <cstdlib>

using namespace std;

class Money {
private:
    long rubles;
    int kopecks;

public:
    Money() : rubles(0), kopecks(0) {}
    Money(long r, int k) : rubles(r), kopecks(k) {
        normalize();
    }

    void normalize() {
        if (kopecks >= 100) {
            rubles += kopecks / 100;
            kopecks %= 100;
        }
        else if (kopecks < 0) {
            long adjust = (-kopecks + 99) / 100;
            rubles -= adjust;
            kopecks += adjust * 100;
        }
    }

    long getRubles() const { return rubles; }
    int getKopecks() const { return kopecks; }

    bool operator==(const Money& other) const {
        return rubles == other.rubles && kopecks == other.kopecks;
    }

    bool operator!=(const Money& other) const {
        return !(*this == other);
    }

    Money& operator=(const Money& other) {
        if (this != &other) {
            rubles = other.rubles;
            kopecks = other.kopecks;
        }
        return *this;
    }

    friend istream& operator>>(istream& is, Money& m) {
        char comma;
        is >> m.rubles >> comma >> m.kopecks;
        if (comma != ',') is.setstate(ios::failbit);
        m.normalize();
        return is;
    }

    friend ostream& operator<<(ostream& os, const Money& m) {
        os << m.rubles << ",";
        if (m.kopecks < 10) os << "0";
        os << m.kopecks;
        return os;
    }
};

template<typename T>
class Set {
private:
    T* elements;

```

```

int size;
int capacity;

bool contains(const T& value) const {
    for (int i = 0; i < size; i++) {
        if (elements[i] == value) return true;
    }
    return false;
}

public:
Set() : size(0), capacity(10) {
    elements = new T[capacity];
}

Set(int cap) : size(0), capacity(cap) {
    elements = new T[capacity];
}

Set(const Set& other) : size(other.size), capacity(other.capacity) {
    elements = new T[capacity];
    for (int i = 0; i < size; i++) {
        elements[i] = other.elements[i];
    }
}

~Set() {
    delete[] elements;
}

Set& operator=(const Set& other) {
    if (this != &other) {
        delete[] elements;
        size = other.size;
        capacity = other.capacity;
        elements = new T[capacity];
        for (int i = 0; i < size; i++) {
            elements[i] = other.elements[i];
        }
    }
    return *this;
}

int operator()() const {
    return size;
}

T& operator[](int index) {
    if (index < 0 || index >= size) {
        throw out_of_range("Индекс вне границ");
    }
    return elements[index];
}

const T& operator[](int index) const {
    if (index < 0 || index >= size) {
        throw out_of_range("Индекс вне границ");
    }
    return elements[index];
}

Set operator-(const Set& other) const {
    Set result;
    for (int i = 0; i < size; i++) {
        if (!other.contains(elements[i])) {
            result.add(elements[i]);
        }
    }
    return result;
}

```

```

    }

    void add(const T& value) {
        if (!contains(value)) {
            if (size >= capacity) {
                capacity *= 2;
                T* newElements = new T[capacity];
                for (int i = 0; i < size; i++) {
                    newElements[i] = elements[i];
                }
                delete[] elements;
                elements = newElements;
            }
            elements[size++] = value;
        }
    }

    friend istream& operator>>(istream& is, Set<T>& s) {
        int n;
        T value;
        cout << "Введите количество элементов: ";
        is >> n;
        for (int i = 0; i < n; i++) {
            cout << "Элемент " << i + 1 << ": ";
            is >> value;
            s.add(value);
        }
        return is;
    }

    friend ostream& operator<<(ostream& os, const Set<T>& s) {
        os << "{ ";
        for (int i = 0; i < s.size; i++) {
            os << s.elements[i];
            if (i < s.size - 1) os << ", ";
        }
        os << " }";
        return os;
    }
};

int main() {
    setlocale(LC_ALL, "Russian");
    system("chcp 1251 > nul");

    cout << " ТЕСТ 1: Set<int> \n";
    Set<int> intSet1, intSet2;

    cout << "Введите множество A (int):\n";
    cin >> intSet1;
    cout << "Введите множество B (int):\n";
    cin >> intSet2;

    cout << "A = " << intSet1 << endl;
    cout << "B = " << intSet2 << endl;
    cout << "Размер A = " << intSet1() << endl;
    cout << "Первый элемент A = " << intSet1[0] << endl;

    Set<int> intDiff = intSet1 - intSet2;
    cout << "A - B = " << intDiff << endl;

    cout << "\n ТЕСТ 2: Set<float> \n";
    Set<float> floatSet1, floatSet2;

    cout << "Введите множество A (float):\n";
    cin >> floatSet1;
    cout << "Введите множество B (float):\n";
    cin >> floatSet2;

```

```

cout << "A = " << floatSet1 << endl;
cout << "B = " << floatSet2 << endl;
cout << "Размер A = " << floatSet1() << endl;

Set<float> floatDiff = floatSet1 - floatSet2;
cout << "A - B = " << floatDiff << endl;

cout << "\n ТЕСТ 3: Set<double> \n";
Set<double> doubleSet1, doubleSet2;

cout << "Введите множество A (double):\n";
cin >> doubleSet1;
cout << "Введите множество B (double):\n";
cin >> doubleSet2;

cout << "A = " << doubleSet1 << endl;
cout << "B = " << doubleSet2 << endl;
cout << "Размер A = " << doubleSet1() << endl;

Set<double> doubleDiff = doubleSet1 - doubleSet2;
cout << "A - B = " << doubleDiff << endl;

cout << "\n ТЕСТ 4: Set<Money> \n";
Set<Money> moneySet1, moneySet2;

cout << "Введите множество A (Money в формате рубли,копейки):\n";
cin >> moneySet1;
cout << "Введите множество B (Money в формате рубли,копейки):\n";
cin >> moneySet2;

cout << "A = " << moneySet1 << endl;
cout << "B = " << moneySet2 << endl;
cout << "Размер A = " << moneySet1() << endl;

if (moneySet1() > 0) {
    cout << "Первый элемент A = " << moneySet1[0] << endl;
}

Set<Money> moneyDiff = moneySet1 - moneySet2;
cout << "A - B = " << moneyDiff << endl;

cout << "\n ТЕСТ 5: Копирование и присваивание \n";
Set<Money> moneyCopy(moneySet1);
cout << "Копия A = " << moneyCopy << endl;

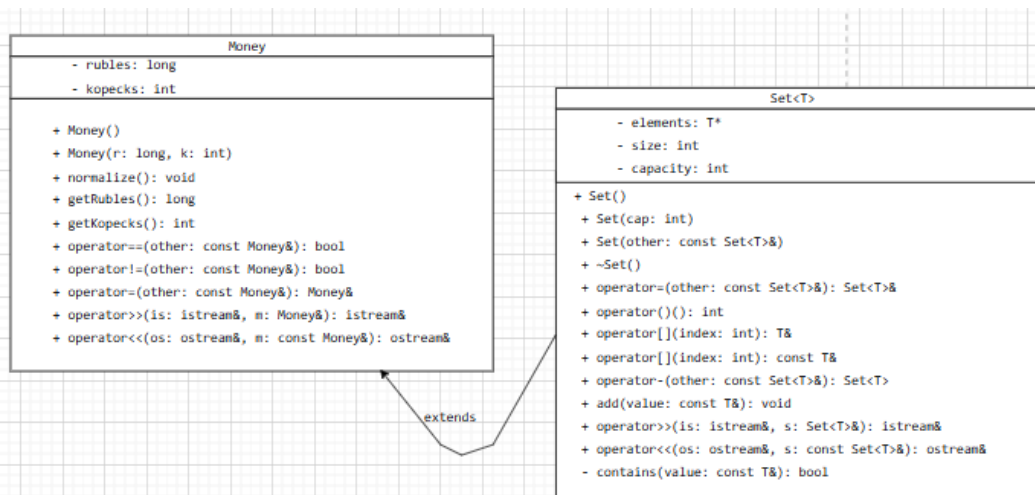
Set<Money> moneyAssign;
moneyAssign = moneySet2;
cout << "Присваивание B = " << moneyAssign << endl;

cout << "\nВсе тесты успешно выполнены!\n";

return 0;
}

```

UML диаграмма



Вывод программы

```
ТЕСТ 1: Set<int>
Введите множество A (int):
Введите количество элементов: 3
Элемент 1: 1
Элемент 2:
4
Элемент 3: 4
Введите множество B (int):
Введите количество элементов: 3
Элемент 1: 5
Элемент 2: 6
Элемент 3: 7
A = { 1, 4 }
B = { 5, 6, 7 }
Размер A = 2
Первый элемент A = 1
A - B = { 1, 4 }

ТЕСТ 2: Set<float>
Введите множество A (float):
Введите количество элементов: 2
Элемент 1: 2.5
Элемент 2: 3.8
Введите множество B (float):
Введите количество элементов: 3
Элемент 1: 3.8
Элемент 2: 8.7
Элемент 3: 6.7
A = { 2.5, 3.8 }
B = { 3.8, 8.7, 6.7 }
Размер A = 2
A - B = { 2.5 }

ТЕСТ 3: Set<double>
Введите множество A (double):
Введите количество элементов: 3
Элемент 1: 5.235
Элемент 2: 4.34556
Элемент 3: 2.24565
Введите множество B (double):
Введите количество элементов: 3
Элемент 1: 2.35
Элемент 2: 1.9084
Элемент 3: 7.90134
A = { 5.235, 4.34556, 2.24565 }
B = { 2.35, 1.9084, 7.90134 }
Размер A = 3
A - B = { 5.235, 4.34556, 2.24565 }

ТЕСТ 4: Set<Money>
Введите множество A (Money в формате рубли,копейки):
Введите количество элементов: 2
Элемент 1: 2.60
Элемент 2: Введите множество B (Money в формате рубли,копейки):
Введите количество элементов: A = { 2,60 }
B = { }
Размер A = 1
Первый элемент A = 2,60
A - B = { 2,60 }

ТЕСТ 5: Копирование и присваивание
Копия A = { 2,60 }
Присваивание B = { }

Все тесты успешно выполнены!
```


Ответы на вопросы

1. Смысл использования шаблонов — написание одного кода для разных типов данных без дублирования. Шаблоны позволяют создавать обобщённые функции и классы.
2. Синтаксис шаблона функции: `template <typename T> T func(T a, T b) { ... }.` Семантика: компилятор генерирует отдельную версию функции для каждого используемого типа.
3. Синтаксис шаблона класса: `template <class T> class MyClass { ... };` Семантика: класс описывает поведение для любого типа, инстанцирование происходит при создании объекта с конкретным типом.
4. Параметры шаблона функции — это типы (`typename T`) или константы (`int N`), которые подставляются при вызове функции. Позволяют функции работать с разными типами данных.
5. Свойства параметров шаблона функции: могут быть типами (`typename`) или нетповыми (константы); могут иметь значения по умолчанию; выводятся из аргументов вызова (для типов); каждый параметр должен быть уникальным.
6. Параметр шаблона записывается в угловых скобках после `template`. Для типа: `typename T` или `class T`. Для нетпового: `int N`. Для шаблона шаблона: `template <typename> class C`.
7. Да, параметризованные функции можно перегружать. Можно перегружать шаблоны между собой (разное количество параметров) и с обычными функциями.
8. Свойства параметризованных классов: инстанцируются с конкретным типом; могут иметь статические члены (разные для каждого типа); не поддерживают отдельную компиляцию (определения в заголовочных файлах); могут быть частично специализированы.
9. Нет, не все. Неконстантные статические члены и конструкторы с деструкторами не параметризованы. Обычные методы могут не зависеть от параметра шаблона.
10. Нет, не являются. Дружественная функция, определённая внутри параметризованного класса, становится обычной нешаблонной функцией, если в её объявлении нет параметров шаблона.

11. Да, могут. Шаблоны классов могут содержать виртуальные функции. При инстанцировании класса виртуальные функции становятся обычными виртуальными функциями для данного типа.

12. Вне определения шаблона класса компонентные функции определяются с синтаксисом: `template <typename T> return_type MyClass<T>::func(args) { ... }`. Шаблонный заголовок повторяется, и после имени класса указывается `<T>`.

13. Инстанцирование шаблона — это процесс генерации компилятором реального кода (функции или класса) для конкретного набора параметров шаблона.

14. Генерирование определения класса по шаблону происходит на этапе компиляции. Конкретнее — при первом использовании шаблона с новыми параметрами (при создании объекта или вызове функции).